

tf.TensorShape Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/TensorShape

Represents the shape of a Tensor.

Function Signatures

```
tf.TensorShape( dims )  
t = tf.constant([[1,2,3],[4,5,6]])
```

Code Examples

```
tf.TensorShape(  
    dims  
)
```

```
tf.TensorShape(  
    dims  
)
```

```
t = tf.constant([[1,2,3],[4,5,6]])  
t.shape  
TensorShape([2, 3])
```

```
t = tf.constant([[1,2,3],[4,5,6]])
```

```
TensorShape([2, 3])
```

```
@tf.function  
def get_dynamic_shape(t):  
    print("tracing...")  
    print(f"static shape is {t.shape}")  
    return tf.shape(t)
```

```
@tf.function
```

Documentation

Represents the shape of a Tensor.

Inherits From: `TraceType`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.TensorShape`

Used in the notebooks

- TensorFlow 1.x vs TensorFlow 2 - Behaviors and APIs
 - `DTensor` concepts
 - Working with RNNs
 - Distributed Input
 - Tensorflow datasets from MongoDB collections

`TensorShape` is the static shape representation of a `Tensor`.

During eager execution a `Tensor` always has a fully specified shape but when tracing a `tf.function` it may be one of the following:

- Fully-known shape: has a known number of dimensions and a known size

for each dimension. e.g. `TensorShape([16, 256])`

- Partially-known shape: has a known number of dimensions, and an unknown

size for one or more dimension. e.g. `TensorShape([None, 256])`

- Unknown shape: has an unknown number of dimensions, and an unknown

size in all dimensions. e.g. `TensorShape(None)`

During function tracing `t.shape` will return a `TensorShape` object representing the shape of `Tensor` as it is known during tracing.

This static representation will be partially defined in cases where the exact shape depends on the values within the tensors. To get the dynamic representation, please use `tf.shape(t)`

which will return `Tensor` representing the fully defined shape of `t`.

This way, you can express logic that manipulates the shapes of tensors by building other tensors that depend on the dynamic shape of `t`.

For example, this function prints the `TensorShape` (`t.shape`), when you trace the function, and returns a tensor [`tf.shape\(t\)`](#) for given input:

Just calling the function traces it with a fully-specified static shape:

But `tf.function` can also trace the function with a partially specified (or even unspecified) shape:

If a tensor is produced by an operation of type "Foo", its shape may be inferred if there is a registered shape function for "Foo". See `Shape` functions

for details of shape functions and how to register them. Alternatively, you may set the shape explicitly using `tf.Tensor.ensure_shape`.

Raises

`## Attributes`

Suggest `TensorShape.as_list` instead.

Methods

`## as_list`

[View source](#)

Returns a list of integers or `None` for each dimension.

as_proto

[View source](#)

Returns this shape as a `TensorShapeProto`.

assert_has_rank

[View source](#)

Raises an exception if `self` is not compatible with the given rank.

assert_is_compatible_with

[View source](#)

Raises exception if `self` and `other` do not represent the same shape.

This method can be used to assert that there exists a shape that both `self` and `other` represent.

assert_is_fully_defined

[View source](#)

Raises an exception if self is not fully defined in every dimension.

assert_same_rank

[View source](#)

Raises an exception if self and other do not have compatible ranks.

concatenate

[View source](#)

Returns the concatenation of the dimension in self and other.

experimental_as_proto

[View source](#)

Returns a proto representation of the TensorShape instance.

experimental_from_proto

[View source](#)

Returns a TensorShape instance based on the serialized proto.

experimental_type_proto

[View source](#)

Returns the type of proto associated with TensorShape serialization.

is_compatible_with

[View source](#)

Returns True iff self is compatible with other.

Two possibly-partially-defined shapes are compatible if there exists a fully-defined shape that both shapes can represent. Thus, compatibility allows the shape inference code to reason about partially-defined shapes. For example:

- `TensorShape(None)` is compatible with all shapes.
- `TensorShape([None, None])` is compatible with all two-dimensional

shapes, such as `TensorShape([32, 784])`, and also `TensorShape(None)`. It is not compatible with, for example, `TensorShape([None])` or `TensorShape([None, None, None])`.

- `TensorShape([32, None])` is compatible with all two-dimensional shapes

with size 32 in the 0th dimension, and also `TensorShape([None, None])` and `TensorShape(None)`. It is not compatible with, for example, `TensorShape([32])`, `TensorShape([32, None, 1])` or `TensorShape([64, None])`.

- `TensorShape([32, 784])` is compatible with itself, and also

`TensorShape([32, None])`, `TensorShape([None, 784])`, `TensorShape([None, None])` and `TensorShape(None)`. It is not compatible with, for example, `TensorShape([32, 1, 784])` or `TensorShape([None])`.

`TensorShape(None)` is compatible with all shapes.

`TensorShape([None, None])` is compatible with all two-dimensional shapes, such as `TensorShape([32, 784])`, and also `TensorShape(None)`. It is not compatible with, for example, `TensorShape([None])` or `TensorShape([None, None, None])`.

`TensorShape([32, None])` is compatible with all two-dimensional shapes with size 32 in the 0th dimension, and also `TensorShape([None, None])` and `TensorShape(None)`. It is not compatible with, for example, `TensorShape([32])`, `TensorShape([32, None, 1])` or `TensorShape([64, None])`.

`TensorShape([32, 784])` is compatible with itself, and also `TensorShape([32, None])`, `TensorShape([None, 784])`, `TensorShape([None, None])` and `TensorShape(None)`. It is not compatible with, for example, `TensorShape([32, 1, 784])` or `TensorShape([None])`.

The compatibility relation is reflexive and symmetric, but not transitive. For example, `TensorShape([32, 784])` is compatible with `TensorShape(None)`, and `TensorShape(None)` is compatible with `TensorShape([4, 4])`, but `TensorShape([32, 784])` is not compatible with `TensorShape([4, 4])`.

`is_fully_defined`

[View source](#)

Returns True iff self is fully defined in every dimension.

`is_subtype_of`

[View source](#)

Returns True iff self is subtype of other.

Shape A is a subtype of shape B if shape B can successfully represent it:

- A `TensorShape` of any rank is a subtype of `TensorShape(None)`.
- `TensorShapes` of equal ranks are covariant, i.e.

`TensorShape([A1, A2, ..])` is a subtype of

`TensorShape([B1, B2, ..])` iff A_n is a subtype of B_n .

A_n is subtype of B_n iff $A_n == B_n$ or B_n is `None`.

- `TensorShapes` of different define